PRN: 260840125033 260840125041

For a Toy manufacturing company, create an app to perform basic operations on the stock

The Toy data has
product id
name
price
category
age [e.g. 4+]
purachase month, year

1. List the Stock
2. Filter the stock by category [e.g. Educational / Battery operated etc]
3. List Toys by price range (e.g. between 500 to 1000)
4. Sort the toys by price- category wise
5. Make the list of old stock [older than 1 year]
6. Make a group of toys as per their category, count them
7. Display the most expensive/least expensive toy in given category
8. Print total no of toys in stock for each age grp

// Toy Class

```
package javaassignment4;

public class Toy {
	private int product_id;
	private String name;
	private double price;
	private String category;
	private String age;
	private int purachase_month;
	private int year;

	public Toy(int product_id, String name, double price, String category, String age, int purachase_month, int year) {
		this.product_id = product_id;
		this.name = name;
		this.price = price;
		this.category = category;
		this.age = age;
		this.purachase_month = purachase_month;
		this.year = year;
	}

	public int getProduct_id() {
```

```
        return product_id;
}

public void setProduct_id(int product_id) {
        this.product_id = product_id;
}

public String getName() {
        return name;
}

public void setName(String name) {
        this.name = name;
}

public double getPrice() {
        return price;
}

public void setPrice(double price) {
        this.price = price;
}

public String getCategory() {
        return category;
}

public void setCategory(String category) {
        this.category = category;
}

public String getAge() {
        return age;
}

public void setAge(String age) {
        this.age = age;
}

public int getPurachase_month() {
        return purachase_month;
}

public void setPurachase_month(int purachase_month) {
        this.purachase_month = purachase_month;
}
```

```
	public int getYear() {
		return year;
	}

	public void setYear(int year) {
		this.year = year;
	}

	@Override
	public String toString() {
		return "Toy [product_id=" + product_id + ", name=" + name + ", price=" + price + ",
category=" + category
				+ ", age=" + age + ", purachase_month=" + purachase_month + ",
year=" + year + "]";
	}
}
```

// ToyInventoryApp Class

```
package javaassignment4;

import java.time.Year;
import java.util.*;
import java.util.stream.Collectors;

public class ToyInventoryApp {
	public static void main(String[] args) {
		List<Toy> stock = new ArrayList<>();
		stock.add(new Toy(1, "Bat", 800, "sports", "5+", 5, 2016));
		stock.add(new Toy(2, "Ball", 6000, "sports", "5+", 12, 2026));
		stock.add(new Toy(3, "Football", 6000, "sports", "2+", 3, 2023));
		stock.add(new Toy(4, "Gun", 6000, "boys", "15+", 5, 2022));
		stock.add(new Toy(5, "Barbie", 400, "girls", "10+", 6, 2025));
		stock.add(new Toy(6, "PS5", 900, "video games", "15+", 9, 2026));

		// 1. List all the stocks
		stock.stream()
		    .forEach(System.out::println);

		// 2. Filter the stock by category
		Scanner sc = new Scanner(System.in);
		System.out.println("Enter the Category: ");
		String category1 = sc.next();
```

```
stock.stream()
    .filter(x -> category1.equals(x.getCategory()))
    .forEach(System.out::println);

stock.stream()
    .filter(e -> e.getCategory().contains("sports"))
    .forEach(System.out::println);

// 3. List Toys by price range (between 500 to 1000)
stock.stream()
    .filter(x -> x.getPrice() >= 500 && x.getPrice() <= 1000)
    .forEach(System.out::println);

// 4. Sort the toys by price- category wise
stock.stream()
    .sorted(Comparator.comparing(Toy::getPrice).thenComparing(Toy::getCategory))
    .forEach(System.out::println);

// 5. Make the list of old stock [older than 1 year]
int currYear = Year.now().getValue();
stock.stream()
    .filter(x -> x.getYear() < currYear)
    .forEach(System.out::println);

// 6. Make a group of toys as per their category, count them
System.out.println(
   stock.stream()
        .collect(Collectors.groupingBy(Toy::getCategory, Collectors.counting()))
);

// 7. Display the most expensive/least expensive toy in given category
Scanner sc2 = new Scanner(System.in);
System.out.println("Enter the Category: ");
String category2 = sc2.next();

stock.stream()
    .filter(a -> category2.equals(a.getCategory()))
    .max(Comparator.comparing(Toy::getPrice))
    .ifPresent(System.out::println);

// 8. Print total no of toys in stock for each age grp
Map<String, Long> ageStockCount = stock.stream()
   .collect(Collectors.groupingBy(Toy::getAge, Collectors.counting()));

ageStockCount.forEach((k, v) -> System.out.println(k + ": " + v));
```

```
            sc.close();
        }
}
```

Concepts Learned:

1. Stream API
Streams help process collections of data without writing explicit loops for every operation. They support filtering, sorting, transforming, and collecting data.

- stream() - Creates a sequential stream from a collection.
- filter(predicate) - Selects elements matching a given condition.
- map(function) - Transforms each element into another value or type.
- flatMap(function) - Flattens nested structures (e.g., lists within lists) into a single stream.
- distinct() - Removes duplicate elements based on equals().
- sorted() - Sorts elements naturally or using a comparator.
- limit(n) / skip(n) - Truncates or skips elements in the stream.
- forEach(consumer) - Performs an action on each element (terminal).
- collect(collector) - Gathers results into a collection or summary (terminal).
- reduce(identity, accumulator) - Combines stream elements into a single result (e.g., sum, product).
- anyMatch(), allMatch(), noneMatch() - Short-circuiting operations to check conditions.
- findFirst(), findAny() - Returns an Optional matching elements.

Stream operations are classified into intermediate operations (which return another stream, like filter(), map(), sorted()) and terminal operations (which trigger processing and close the stream, like collect(), forEach(), reduce()). Processing is lazy, meaning intermediate operations do not execute until a terminal operation is invoked.

2. Method References
Method references use the :: operator to refer to an existing method concisely when a lambda expression only calls that method.

There are four primary types of method references:

1. Reference to a static method
2. Reference to an instance method of a particular object: (e.g., System.out::println)

3. Reference to an instance method of an arbitrary object of a particular type: (e.g., String::toUpperCase)
4. Reference to a constructor: (e.g., ArrayList::new)

3. Comparator
Comparator is used to define custom sorting logic for objects based on their properties.

Key features and methods:

- comparing() / thenComparing() - Chains multiple sorting criteria.
- comparingInt(), comparingLong(), comparingDouble() - Primitive-specialized variants to avoid autoboxing overhead. (Autoboxing like from int to Integer)
- reversed() - Reverses the sort order.
- Comparator.nullsFirst() / Comparator.nullsLast() - Handles null values gracefully during sorting.

4. Optional
Optional represents a container object that may or may not contain a non-null value, eliminating the need for explicit null checks and avoiding NullPointerException.

Common methods:

- isPresent() / isEmpty() - Checks if a value is present.
- ifPresent(consumer) - Executes an action if the value exists.
- orElse(other) - Returns the value if present, otherwise returns a default value.
- orElseGet(supplier) - Returns the value or computes a fallback value lazily.
- orElseThrow() - Returns the value or throws an exception if empty.
- map(function) / filter(predicate) - Transforms or filters the value inside the Optional if present.

5. Collectors and Grouping
Collectors provides a rich set of predefined reduction operations for streaming data.
groupingBy() categorizes elements based on a classification function and stores them in a Map.

Other Useful Collectors

- Collectors.toList() / Collectors.toSet() - Collects elements into a List or Set.
- Collectors.joining(", ") - Concatenates String elements with a delimiter.
- Collectors.summingInt(), Collectors.averagingDouble() - Computes sums and averages.
- Collectors.mapping() - Adapts collectors to map elements before collecting them.